DSH 会话上下文瘦身完整报告

__版本__：1.0 · 2026-10-23（首版）· 2026-10-26 整理为对外报告

__作者__：DSH 用户（匿名）

__适用__：DeepSeek Harness（DSH）desktop，会话格式 v4，Windows

__对象__：某工作区的一个 QQ 会话（onebot-private-<QQ 号>-<随机会话后缀>）

__一句话__：把一个 3.47MB / 12141 事件、模型上下文约 87 万 token 的会话，物理裁剪成 331KB / 789 事件、约 5 万 token，且完整保留「摘要 + 最近 60 轮对话 + 权限状态」，会话可正常恢复、继续对话。

目录

1. 成果速览
2. 问题背景
3. 原理篇：DSH 会话到底是怎么存的
4. 方案设计：裁剪策略
5. 实践篇：完整操作步骤
6. 踩坑大全（9 个坑）
7. 记忆插件：为什么它让瘦身变安全
8. 效果数据
9. 局限与非适用场景
10. 附录

1. 成果速览

指标	瘦身前	瘦身后	比例
会话文件大小	3,465,669 字节（3.47MB）	331,914 字节（331KB）	9.6%
事件数	12,141	789	6.5%
对话 turn 数	667	60	9%
模型上下文（消息字符）	约 3.5MB / 3543 条消息	约 300KB / 295 条消息	—
模型上下文（估算 token）	约 874,000	约 50,000（contextPressure.surfaceTokens=50,201）	5.7%
权限状态	danger-full-access / never	**原样保留**	✅

会话标题 / 预设 / 工作目录	原样保留	**原样保留**	✅
恢复后 inbox	有 pending 队列	空（安全）	✅

代价：某日全天（234 条 user message）与更早的原始对话被压成**一条摘要**。原文完整保留在 3 份备份 + 用户自导文本里（见第 6 章）。

2. 问题背景

2.1 现象

- QQ 端 AI 助手（dsh-onebot-qq 插件 → agent）的会话日志持续增长到 3.47MB、12141 个事件、667 轮对话。
- 每次模型请求都要重放大段历史，上下文约 87 万 token（1M 窗口的 87%），token 成本和首字延迟都很高。
- 会话内官方压缩（compaction）已触发过 9 次（compaction/prune），但**官方压缩只遮蔽、不删除**（见 3.6），文件不会变小，历史事件仍在。

2.2 目标

物理裁剪会话日志：**保留最近 60 轮完整对话 + 一条摘要，其余删除**，同时保证：

11. 会话能被 DSH 正常加载（三段校验全过）；
12. 恢复后权限/标题/预设等状态不丢；
13. AI 助手"记得"重要的东西（靠摘要 + 记忆文件）；
14. 全程可回滚。

2.3 约束

- 会话文件被 QQ 插件**常驻内存持有**，直接替换会被覆盖，必须让插件先释放（/new 或重启）；
- DSH 对会话格式有严格校验，删错一行就报 corrupt；
- 不同事件类型语义不同：有的进模型上下文，有的不进；有的有状态、有的无状态。

3. 原理篇：DSH 会话到底是怎么存的

这一章是"为什么这么删才安全"的基础。理解这四层，才能改对。

3.1 物理格式：zstd 帧拼接

会话文件路径（Windows）：

```
%USERPROFILE%\.dsh\sessions\--<工作区路径转义>--\<会话 id>\session.v4.jsonl.zstd
```

物理编码（dsh-session-persistence-jsonl）：

- 文件 = **多个独立 Zstandard 帧的标准拼接**；
- **第一帧恰好等于一行 header**（{"type":"session","version":4,"id":...,"createdAt":...,"isSeeded":false,"delegationDepth":0,"cwd":...}），且这一帧必须只含这一行；
- 后续每帧 = 一个写入批次（多行 JSON 事件）；
- **所有帧都带 checksum**（ZSTD_c_checksumFlag=1），读取时逐帧校验；
- 帧内记录必须完整（不能跨帧撕裂一行 JSON）；
- 读取时把各帧解压、拼接成事件流。

实测：原文件 628 个帧 / 12142 行（header 1 行 + 事件 12141 行），平均每帧约 19 行。

3.2 事件模型（v4）

会话 = 仅追加的事件日志。每行一个 JSON 事件：

```
{"type":"user/message","seq":<seq>,"time":<timestamp>,
 "data":{"content":[{"type":"text","text":"..."}],
         "source":{"kind":"user","rpcId":"...","clientTimeZone":"Asia/Shanghai"},
         "role":"user","id":"..."},
 "surfaceOp":"append"}
```

事件分类（决定"删不删、怎么删"）：

类别	事件类型	是否进模型上下文	裁剪时能不能删
surface（模型可见）	system/message、user/message、developer/message、assistant/message、tool/result	✅ 进	能删，但必须整段 turn 删，保住结构
生命周期	turn/start、turn/end、step/start、step/end	❌ 不进（结构）	不能单独删，必须成对保留
工具调用	tool/call	❌ 不进（与 result 配对）	与 tool/result 成对处理
运行时日志	agent/inbox/spliced、job/registered、job/settled、workspace/changes	❌ 不进	可整类删（有坑，见 6.1）
请求/状态	request/header、request/context、	部分进	状态类要"抢救最新一条"

	session/title、permission/preset、sandbox/mode、approval/policy、session/end-seed		
压缩	compaction/start、compaction/summary、compaction/end、compaction/prune	摘要进	裁剪时整组删（摘要文本取出重用）

surfaceOp 是 surface 事件的操作语义：

- "append"：追加到 surface（正常消息）；
- {"op":"replace","startSeq":8,"endSeq":6651}：**替换**（遮蔽）surface 上的一段（压缩/修剪工具结果用），并带 sourceEventSeqs 列出被遮蔽的节点。

关键事实：压缩（compaction）和工具结果修剪（pruner）**都是 replace 操作，不删原事件**——所以官方压缩不会让文件变小，只是让 deriveMessages() 少读一些。

3.3 模型上下文是怎么来的

- Session.deriveMessages() 把事件日志**投影（fold）**成模型看到的 Message[]；
- 只看 surface 事件，按 surfaceOp 折叠（append 加入 / replace 替换段）；
- assistant/message 的 data.message.content 里含 reasoning / text / tool-call 块，replayState.blocks 要与 content 对齐（**不要手改 assistant 消息内容**）；
- tool/result 通过 data.message.source.callId 与 tool/call 的 data.callId 配对。

3.4 加载校验三层（改错任何一层 = 会话打不开）

DSH 读取会话时依次校验：

第一层：seq 连续性（assertContiguous）

- 事件 seq 必须从 0 开始、连续递增、无断号无重复。
- 报错示例：released v2 row ... has seq gap (expected 10677, got 10699)。

第二层：v4 生命周期关系（assertReleasedV4Relationships，v3-to-v4 包里的 Relationships 类）

- turn/start 的 data.turn 必须等于 nextTurn（从 1 开始，turn/end 后 +1）→ **turn 编号必须连续从 1 起**；

- step/start 的 data.step 必须等于当前 turn 内的 nextStep（每 turn 从 1 起）；
- step/end 必须匹配 open 的 turn+step；
- turn/end 要求当前无 open step；closeTools 要求没有未闭合的 tool call；
- assistant/message 里每个 tool-call 必须与后续 tool/call（同 id、同 name/arguments）、tool/result（同 callId）配对；
- tool/result 若为 replace 操作，要求 open turn；
- surface 折叠：
- **第一条 surface 必须是 `system/message`**（成为"受保护头"protectedHead），否则后续 system append 报 system/message requires a protected first surface head；
- 替换事件 sourceEventSeqs 必须覆盖被遮蔽的 surface 节点；
- 替换不能遮蔽 protectedHead（除非替换事件本身是恰好覆盖它的 system/message）。
- developer/message 的 headerSeq 必须指向更早的 request/header；
- session/title 的 messageSeqs 必须引用更早的、source.kind==='user' 的 user/message；
- compaction/* 必须成对/匹配（start→summary→end，prune 的 shadowedSeqs 必须等于当前 surface 段）；
- session/end-seed 会重置 compaction 状态。

第三层：消息 schema（assertV4RowAdmission / adoptSessionEvent）

- assistant/message 的 source 必须是 {"kind":"model","provider":...,"model":...}；
- tool/result 的 source 必须是 {"kind":"tool","callId":...}，且 message.toolCallId 匹配；
- 消息必须有 source.kind（非空、非 'plugin'）、id、role、content 数组；
- sourceEventSeqs 非空（替换事件）；
- 事件类型必须在 KNOWN_SESSION_EVENT_TYPES（62 种）内。

第四层（插件侧）：投影 fold（session projection ...）

- 投影缓存/恢复会把事件 fold 成各投影状态（inbox、permissions、title、modelSelection...）；
- fold 时对 seq 连续性、inbox splice 语义有单独校验（见 6.1 坑）。

校验最快的方式：直接 import asar 里的真实模块跑，而不是自己复刻规则（见 5.4）。

3.5 投影缓存（session_projcache）

路径：

```
%USERPROFILE%\.dsh\storages\session_projcache\sessions\<会话 id>.json
```

- 每个会话一份检查点，记录各投影（inbox、permissions、title、turnBoundary、sessionStats、tokenUsage、contextPressure、turnOutline…）的 {ver, seq, val}；
- seq 是"已 fold 到的事件水位"，恢复时从该水位之后继续 fold，跳过前缀；
- **缓存永远不领先于日志**（日志权威）。但如果你**物理删了文件里的事件**，旧缓存的 seq=12140 就"领先"于新日志（789 事件）——投影会以为前缀都处理过了，直接跳过 → 状态错乱。
- **结论：裁剪后必须删除/移走该缓存**，让它冷 fold 重建。

3.6 官方压缩 vs 物理裁剪

维度	官方 compaction / pruner	物理裁剪（本报告）
机制	追加 replace 事件遮蔽旧 surface	删除旧事件、重排 seq、重写文件
文件大小	不变（只增不减）	大幅减小
模型上下文	变小（遮蔽生效）	变小
触发方式	自动阈值 / /compact	手工离线操作
风险	低（官方路径）	中高（需严格校验）
适用	日常长会话	占用过大、想彻底瘦身

4. 方案设计：裁剪策略

4.1 核心思路

只按整段 turn 裁剪，绝不拆 turn 内部结构：

15. 选定保留起点（第 K 个 turn 的 turn/start）；
16. 保留从该起点到文件末尾的所有事件（去掉要删的冗余类别）；
17. 在保留区间开头插入：
18. 一条 system/message（复制最后一条系统提示，作为 protectedHead）；
19. 一条 user/message 摘要（<compacted-summary>，内容取自原文件 compaction/summary）；
20. 把保留区间内的旧 turn 编号（如 608..667）重排为 1..60；
21. 全部事件 seq 重排为 0..N-1；
22. 重映射所有引用（sourceEventSeqs、messageSeqs、headerSeq）；
23. 抢救"最新状态事件"（权限/沙箱/审批）到文件头部；
24. zstd 帧写回（首帧单行 header + 带 checksum）。

4.2 保留 / 删除 / 抢救 清单

保留（整段 turn 内的）：

- turn/start、turn/end、step/start、step/end
- user/message、assistant/message、tool/call、tool/result
- request/header、developer/message、session/title（最新一条）
- workspace/changes（无害）
- 最后一条 session/end-seed

删除（非模型可见的运行时日志）：

- agent/inbox/spliced（全部，原因见 6.1）
- job/registered、job/settled（全部）
- 保留区间外的所有旧事件
- 保留区间内非最后的 session/end-seed
- compaction/*（摘要文本先提取出来重用）

抢救到文件头部（各取最后一条，否则状态回退默认值）：

- permission/preset（如 {"preset":"danger-full-access"}）
- sandbox/mode（如 {"mode":"danger-full-access"}）
- approval/policy（如 {"policy":"never"}）

重写：

- 所有 data.turn（旧 turn 号 → 新 turn 号，1..K）
- 所有事件 seq（→ 0..N-1）
- 引用字段：sourceEventSeqs、data.messageSeqs、data.headerSeq

不动：

- assistant/message 的 message.content / replayState（原样保留，避免 provider 重放失败）
- tool/result 的 message（原样）
- header（原样，仅校验时去掉 type 字段）

4.3 为什么 turn 号必须重排为 1..K

v4 的 turn/start 校验是 data.turn === this.nextTurn，nextTurn 从 1 开始递增。保留的 turn 若仍是 608..667，第一个 turn/start(608) 就会报 does not open the expected turn。同理 step 编号在每个 turn 内天然从 1 起，**不需要改。**

5. 实践篇：完整操作步骤

以下步骤在 Windows + DSH desktop 实测通过。脚本模板见附录。

5.0 前置：确认环境

25. DSH 用 Electron 内置 Node 24 跑脚本（ELECTRON_RUN_AS_NODE=1）：

```powershell $env:ELECTRON_RUN_AS_NODE='1' & "%LOCALAPPDATA%\Programs\DeepSeek Harness\DeepSeek Harness.exe" --expose-internals script.cjs

`

26. Node 24 自带 zstd：zlib.zstdDecompressSync / zlib.zstdCompressSync。

27. 读 asar 内模块（校验器）：

28. 用 file:// URL 动态 import（Windows 绝对路径必须转 URL）；

29. 读 asar 原始字节时 process.noAsar = true。

```js const { pathToFileURL } = require('node:url'); const v34 = await import(pathToFileURL('<DSH 安装目录>\\resources\\app.asar\\dsh\\node_modules\\@deepseek-ai\\dsh-session-format-v3-to-v4\\lib\\index.js').href);

`

### 5.1 备份（永远先做）

```
$src = "%USERPROFILE%\.dsh\sessions\--<工作区 slug>--\<会话 id>\session.v4.jsonl.zstd"
$bk = "%USERPROFILE%\Documents\deepseek-harness\default-workspace\.tmp-session-slim\backup"
New-Item -ItemType Directory -Force $bk | Out-Null
Copy-Item $src "$bk\session.v4.jsonl.zstd.orig"
Copy-Item "%USERPROFILE%\.dsh\storages\session_projcache\sessions\<会话 id>.json"
"$bk\session_projcache.json.orig"
Copy-Item "%USERPROFILE%\.dsh\media\onebot\retired-sessions.json" "$bk\retired-sessions.json.orig"
(Get-FileHash $src -Algorithm SHA256).Hash   # 记录原文件哈希
```

同时确认聊天备份存在（QQ 端 agent 的 backup_sessions.cjs 会全量备份到 D:\backup\聊天备份\），双保险。

5.2 分析（先看清再动手）

统计：帧数、事件类型分布、seq 范围、turn 分组体积、状态事件位置、引用悬空。 关键脚本：analyze.cjs（类型分布）、turnstats.cjs（每 turn 体积）、stateevents.cjs（状态事件）、refcheck.cjs（引用完整性）、samples.cjs（结构抽样）、byday.cjs（按天消息数）。

分析时要确认：

- seq 是否连续（原文件应 0..N-1 无断号）；

- 保留区间内有没有 replace 操作（有的话要小心 sourceEventSeqs）；
- 保留区间内 agent/inbox/spliced、job/* 的分布；
- 状态事件（permission/sandbox/approval）的最新一条在哪。

5.3 生成（rewrite.cjs 核心逻辑）

30. 解析帧 → header + events；
31. keepFrom = 第 (总数-K) 个 turn/start 的 seq；
32. 保留 seq >= keepFrom 的事件；
33. 过滤掉 agent/inbox/spliced、job/registered、job/settled；
34. session/end-seed 只留最后一条；
35. 抢救 permission/preset、sandbox/mode、approval/policy 各自最后一条，放到最前；
36. 构造 system/message（复制最后一条系统提示，turn=1/step=1）和摘要 user/message，插在保留区间第一个 step/start 之后；
37. turn 重编号（跳过插入事件）；
38. seq 重排（oldToNew 映射）；
39. 重映射引用；
40. 校验（5.4）；
41. 编码写回（5.5）。

摘要内容：取原文件 compaction/summary 的 data.summary 文本（[{type:'text', text}]），外面包官方检查点前导 + <compacted-summary> 标签：

```
This is an automatically generated checkpoint condensing an earlier span of the conversation to free up
context. Treat the captured context as established background and build on it without restating it.
Continue the task directly from the messages that follow, without acknowledging this checkpoint.

<compacted-summary>
...原 summary 文本...
</compacted-summary>
```

摘要事件格式：

```
{"type":"user/message","seq":N,"time":T,
 "data":{"content":[{"type":"text","text":"..."}],
         "source":{"kind":"user","rpcId":"<uuid>","clientTimeZone":"Asia/Shanghai"},
         "role":"user","id":"<uuid>"},
 "surfaceOp":"append"}
```

system/message 格式（复制最后一条的 data.message，改 turn/step）：

```
{"type":"system/message","seq":N,"time":T,
 "data":{"turn":1,"step":1,"message":{...原系统提示...}},
 "surfaceOp":"append"}
```

5.4 校验（必须用 DSH 自己的模块）

```
const v34 = await import(pathToFileURL(ASAR + '\\@deepseek-ai\\dsh-session-format-v3-to-v4\\lib\
\index.js').href);
const ses = await import(pathToFileURL(ASAR + '\\@deepseek-ai\\dsh-session\\lib\\index.js').href);
const KNOWN = ses.KNOWN_SESSION_EVENT_TYPES;

v34.assertReleasedV4Header(logicalHeader);              // 注意：逻辑 header 要去掉 type 字段
for (const e of events) v34.assertV4RowAdmission(e, KNOWN);
v34.assertReleasedV4Relationships({ header: logicalHeader, events, inheritedEventCount: 0 }, KNOWN);

// 完整读取入口（更强）：
v34.restoreReleasedV4Artifact({ header: logicalHeader, events, inheritedEventCount: 0 }, KNOWN);
```

restoreReleasedV4Artifact 是 DSH 读取器实际调用的入口（含 header/行/关系/继承校验），**它 PASS 才算格式有效。**

再自检：

- 首帧解压后恰好一行且 type==="session"；
- 回读行数 = 事件数 + 1；
- 逐事件 seq = 下标；
- 无 agent/inbox/spliced 残留。

5.5 编码写回

```
const CHK = { params: { [zlib.constants.ZSTD_c_checksumFlag]: 1 } };
const parts = [zlib.zstdCompressSync(Buffer.from(JSON.stringify(header) + '\n'), CHK)]; // 首帧单行
for (let i = 0; i < events.length; i += 100) {
  const text = events.slice(i, i + 100).map(e => JSON.stringify(e) + '\n').join('');
  parts.push(zlib.zstdCompressSync(Buffer.from(text), CHK));
}
fs.writeFileSync(out, Buffer.concat(parts));
```

5.6 释放会话写入窗口

插件（dsh-onebot-qq）内存里持有会话对象，直接替换会被覆盖。两种方式：

方式 A：`/new` 切走（推荐，DSH 不用重启）

42. 在 QQ 端给 bot 发 /new（admin 限定）；

43. 插件 resetChat → dispose agent、把旧会话 soft-retire（进 /session 可切换列表），文件不再被写；

44. 替换文件 + 清投影缓存；

45. 发 /session 1 切回，插件重新 open 文件（读瘦身版）。

> ⚠ *若切回**失败**，插件会 hard-retire 会话（见 6.6），恢复要改状态文件 + 重启。*

方式 B：重启 DSH

46. 退出 DSH；

47. 替换文件 + 清缓存；

48. 重新打开 DSH → loadMapping 按绑定表 resume。

5.7 替换 + 清理

```
Copy-Item 瘦身版 $dst -Force
Move-Item "$projcache" "$projcache.bak-$stamp"    # 旧缓存必须移走
```

5.8 状态文件（必要时）

- %USERPROFILE%\.dsh\media\onebot\chat-sessions.json：chatId → 当前 session id 绑定，**保持指向目标会话**；
- %USERPROFILE%\.dsh\media\onebot\retired-sessions.json：retired id 列表，**目标 id 必须不在其中**；
- %USERPROFILE%\.dsh\media\onebot\switchable-sessions.json：/session 列表。

5.9 重启后验证

看投影缓存是否被重建、投影值是否正确：

```
$pc = "%USERPROFILE%\.dsh\storages\session_projcache\sessions\<会话 id>.json"
Get-Content $pc -Raw | ConvertFrom-Json    # 或直接读文本
```

验证点：

- 所有投影 seq 一致且 = 最后事件 seq（如 788）；
- permissions.val = {preset:"danger-full-access", sandbox:"danger-full-access", approval:"never", seeded:true}；
- inbox.val = {"next-turn":[],"next-step":[]}；
- turnBoundary.val.lastTurn = 60、openTurnStartSeq=null（完整收尾）；
- sessionStats.val.turns = 60；
- contextPressure.val.surfaceTokens 明显下降；
- turnOutline.val.turns 数量正确、内容完整。

5.10 回滚

```
# rollback.ps1 核心
Copy-Item backup\session.v4.jsonl.zstd.orig $dst -Force
# 恢复 projcache（从 .bak）
Copy-Item "$projcache.bak-xxx" $projcache -Force
# 之后重启 DSH / 让插件重新 resume
```

6. 踩坑大全（9 个坑）

这一章是本报告最有价值的部分。每个坑都附现象、根因、解法。

坑 1：`invalid persisted inbox splice at session seq 1`（最核心的坑）

现象：第一次切回（/session 1）报：

❌ 切回历史会话失败：`invalid persisted inbox splice at session seq 1 (cause: invalid inbox splice)`
已回退：下一条消息将开启全新会话，原会话仍保留在 `/session` 列表中。

根因：agent/inbox/spliced 是**有状态的收件箱操作日志。**恢复时 inbox 投影从空状态开始，按顺序重放 splice 事件：

```
// dsh-agent-loop 的 inboxProjectionDefinition.apply：
const removedCount = splice.removedCount ?? 0;
if (splice.start + removedCount > inbox.length) throw new Error("invalid inbox splice");
```

裁剪保留了区间中间的 splice 事件，但把**更早的** splice 事件删了 → 重放第一个保留的 splice 时 inbox 为空，start+removedCount > 0 → 报错。

解法：**`agent/inbox/spliced` 必须整类删光。**恢复时 inbox 从空 init，没有 splice 历史 → 合法。这些事件不是模型可见内容，删了不影响对话。

坑 2：`system/message requires a protected first surface head`

现象：校验器报错（或恢复时）。

根因：surface 折叠要求第一条 surface 事件是 system/message（成为 protectedHead）；否则后续 system append 报错。裁剪后如果第一条 surface 是 user/message（摘要），之后保留区间里的 system/message append 就会失败。

解法：在保留区间最前面插入一条 system/message（复制最后一条系统提示，turn=1/step=1，且必须在某 step 内——插在第一个 step/start 之后）。

坑 3：`turn/start does not open the expected turn`

根因：turn 编号必须从 1 连续。保留原 turn 608..667 不合法。

解法：turn 重排为 1..K；所有带 data.turn 的事件（turn/start、turn/end、step/*、assistant/message、tool/result、system/message、developer/message、workspace/changes）同步改写。step 编号不变。

坑 4：权限回退（danger-full-access → workspace-write）

现象：恢复后权限变成默认 workspace-write / ask。

根因：permission/preset、sandbox/mode、approval/policy 是状态事件，最新一条在早期（seq 427-429），被裁掉后投影 fold 不到 → 回退 init 默认值。

解法：抢救这三类事件的**最后一条**，放在新文件头部（turn/start 之前）。它们不在关系校验的 RELATIONSHIP_TYPES 里，放前面安全。

坑 5：投影缓存 asOfSeq 领先

现象：恢复后投影状态错乱/为空。

根因：session_projcache 记录的 seq（如 12140）领先于新日志（789 事件），投影恢复时以为前缀都处理过，跳过事件。

解法：**裁剪后删除/移走 `<会话 id>.json` 投影缓存**，让它冷 fold 重建。

坑 6：`/session` 切回失败 → hard-retire

现象：切回失败后，switchable-sessions.json 里的目标会话消失，retired-sessions.json 里出现目标 id；/session 再也切不回去。

根因：插件 switchSession 失败路径会 retireSession(target)（hard-retire：内存 + 磁盘），从 switchable 列表移除。

解法：

49. 修好文件（删 splice、补状态事件、重校验）；
50. 从 retired-sessions.json 移除目标 id（先备份）；
51. **重启 DSH**（loadMapping 才会重新读 retired 集合 + resume 绑定表里的 session）。

坑 7：把瘦身文件当输入再跑一遍

现象：FAIL: Cannot read properties of undefined (reading 'data')，或"keep from seq 0"。

根因：替换后误把**已瘦身**的文件当原文件再跑 rewrite（瘦身版里没有 compaction/summary，且 turn 数不足 K）。

解法：重试时输入必须用 backup\session.v4.jsonl.zstd.orig（原版）。

坑 8：PowerShell 脚本执行策略 / 中文编码

现象：

- 无法加载文件 ...\.ps1，因为在此系统上禁止运行脚本（ExecutionPolicy）；
- 无 BOM UTF-8 脚本 + 中文导致解析错误（意外的标记 }）。

解法：

- Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force（进程级，不改全局）；
- 脚本避免中文注释/直接用命令行执行；或脚本存 UTF-8 with BOM。

坑 9：删过头

现象：用户反馈"删得有点狠"——某日全天的原始对话（234 条 user message）从会话文件里删了。

根因：保留最近 60 轮 = 覆盖当天，前一天被整个压成摘要。这是"保留 60 轮"选项的边界，不是 bug，但**动手前没跟用户讲透边界。**

解法 / 教训：

52. 动手前明确告知"保留多少轮 / 从哪天起 / 哪些日期会被压缩成摘要"，让用户确认；
53. 备份永远先行（本次有 3 份完整会话备份 + 用户自导的纯文本，内容没丢）；
54. 不可逆裁剪宁可多留几轮。

7. 记忆插件：为什么它让瘦身变安全

这是本次瘦身中同样重要的一环。会话上下文是"短期记忆"，记忆插件是"长期记忆"，两者互补。

7.1 dsh-auto-memory 是什么

- 插件：dsh-auto-memory（本机 v0.9.0）
- 安装位置：%USERPROFILE%\.dsh\profiles\desktop\node_modules\dsh-auto-memory
- 由 desktop profile 的 package.json dependencies + dsh.profile.bundles 引入，自带 cordis.patch.yml 插入 id: auto-memory
- 本机配置走默认：maxBytes 4096、autoSummarize 关闭、enableUserScope true

7.2 磁盘布局

记忆根：%USERPROFILE%\.dsh\memory\

```
memory/
  --<工作区 slug>--/          # project 作用域（按工作区隔离）
    MEMORY.md                 # 索引（自动注入到上下文，约 4KB 预算）
    *.md                      # 单条记忆文件
  _user/                      # user 作用域（全局共享，本机约定不写）
  audit.jsonl                 # 审计日志
```

本机 QQ 端记忆实际落点： %USERPROFILE%\.dsh\memory\--<工作区 slug>--\ （与工作区说明里的 qq-chat\memory 有偏差，以实际为准）

QQ 端 AI 助手当前记忆文件示例：

- identity.md（人格/身份）
- user-profile.md（用户画像）
- feedback-chat-style.md（对话风格反馈）

- project-memory-setup.md（记忆体系）
- reference-sticker-library.md（表情包库）
- project-eyes-ears-tool.md、model-hallucination-flash.md、outbound-send-issue.md（工具/问题记录）

7.3 记忆 vs 上下文（为什么要分开想）

	会话上下文（session 日志）	**记忆文件（auto-memory）**
载体	session.v4.jsonl.zstd	%USERPROFILE%\.dsh\memory\--<工作区 slug>--*.md
内容	原始对话流（user/assistant/tool）	提炼后的长期事实（人设、偏好、项目、教训）
生命周期	可被裁剪/压缩	长期保存，自动注入索引
体积	大（MB 级）	小（KB 级）
瘦身影响	**会被裁剪**	**不受影响**

7.4 本次瘦身中记忆插件起的作用

55. **"不会真的失忆"的底气**：AI 助手的长期记忆（人设、用户偏好、表情包库、项目状态）都在 ~/.dsh/memory/ 下，会话文件被裁剪不影响这些文件。
56. **摘要之外的补充**：摘要由会话内的 compaction/summary 生成，覆盖较早历史；记忆文件则覆盖"长期不变的事实"。两者结合，AI 助手恢复后既记得"最近聊了什么"（摘要+60 轮），也记得"我是谁、用户是谁"（记忆文件）。
57. **瘦身前的检查清单**：动手前应确认重要信息已沉淀到记忆文件/日记/交接报告。本机还有：
58. 日记：D:\backup\日记\<日期>.md
59. 交接报告：qq-chat\交接报告-<日期>.md
60. 对话导出：D:\backup\对话记录\<日期>-完整对话.md

7.5 操作建议（给其他人）

61. **瘦身前**：先让 agent 把当前重要状态写进记忆（memory_write）+ 日记/交接报告；
62. **瘦身后**：确认 MEMORY.md 索引仍能注入（不依赖会话文件）；
63. **记忆作用域**：本机约定只用 project 作用域（user 层不写），避免跨工作区串味；
64. **不要混淆**：裁剪会话不会删记忆文件；但要警惕"记忆里没写、只在会话里的东西"——那才是真正会丢的。

8. 效果数据

8.1 前后对比

项	值
原文件	3,465,669 B / 12141 事件 / 667 turn
新文件	331,914 B / 789 事件 / 60 turn
原模型上下文	约 3.5MB 字符 / 3543 条消息 / ≈874k token
新模型上下文	约 300KB 字符 / 295 条消息 / ≈50k token（surfaceTokens=50,201）
新文件事件类型	turn/start×60、step/start×137、system/message×3、user/message×77、assistant/message×137、tool/call×77、tool/result×77、step/end×137、turn/end×60、workspace/changes×11、request/header×7、session/title×1、developer/message×1、session/end-seed×1、permission/preset×1、sandbox/mode×1、approval/policy×1

8.2 保留量对照（可选 K 值）

保留轮数	文件大小	约 token	适用
K=60（本次）	331KB	≈50k	保留当天完整对话
K=30	95KB	≈30k	只保留最近半天
K=20	78KB	≈18k	只保留最近几小时

9. 局限与非适用场景

65. **不可用于"正在写"的会话**：必须先让插件释放（/new 或重启），否则替换会被内存中的旧会话覆盖。
66. **不能单独删 turn 内部事件**（如只删 step/start）：会破坏关系校验。
67. **不要手改 assistant/message 的 content/replayState**：provider 重放可能失败。
68. **投影缓存必须清理**：否则状态错乱（坑 5）。
69. **官方 compaction 更安全但文件不减**：日常长会话优先用 /compact；本报告适用于"文件太大想彻底瘦身"。
70. **Windows 环境专用**：路径/脚本以 Windows 为例；其他平台思路相同，命令不同。
71. **版本敏感**：v4 格式与校验规则随 DSH 版本可能变化；操作前确认 SESSION_FORMAT_VERSION 与校验器版本。

10. 附录

10.1 工具脚本清单（本次留档）

目录：%USERPROFILE%\Documents\deepseek-harness\default-workspace\.tmp-session-slim\

脚本	用途
analyze.cjs	帧/事件类型/seq 分布统计
turnstats.cjs	按 turn 分组体积统计
stateevents.cjs	权限/沙箱/审批等状态事件定位
refcheck.cjs	引用（sourceEventSeqs/messageSeqs/headerSeq）完整性
samples.cjs	各类事件结构抽样
byday.cjs	user/message 按天分布
overview.cjs	头部/尾部/surfaceOp 概览
tail.cjs	尾部事件与 turn 边界
rewrite.cjs	**核心**：裁剪 + 重排 + 校验 + 写回
codec-check.cjs	用 restoreReleasedV4Artifact 做完整校验
derive-check.cjs	模拟 surface 折叠，查看模型将看到的消息
replace-slim.ps1	替换 + 清缓存 + 修状态（一键）
rollback.ps1	回滚到 orig

10.2 校验 API 速查

```
// asar: @deepseek-ai/dsh-session-format-v3-to-v4/lib/index.js
assertReleasedV4Header(logicalHeader)
assertV4RowAdmission(event, KNOWN_SESSION_EVENT_TYPES)
assertReleasedV4Relationships({ header, events, inheritedEventCount: 0 }, KNOWN)
restoreReleasedV4Artifact({ header, events, inheritedEventCount: 0 }, KNOWN)   // 最强

// asar: @deepseek-ai/dsh-session/lib/index.js
KNOWN_SESSION_EVENT_TYPES   // Set · 62 种
SESSION_FORMAT_VERSION
```

*注意：artifact.header 是**逻辑 header**（无 type 字段）；写文件时 header 行必须带 "type":"session"。*

10.3 备份清单（本次）

备份	路径	大小
原会话	default-workspace\.tmp-session-slim\backup\session.v4.jsonl.zstd.orig	3,465,669 B
替换前快照	同目录 session.v4.jsonl.zstd.before-replace-<时间戳>	3,465,669 B
投影缓存原文件	同目录 session_projcache.json.orig	207,898 B
retired 状态原文件	同目录 retired-sessions.json.orig	—
聊天备份快照	D:\backup\聊天备份\<时间戳>\	各 3.46MB 会话
用户自导文本	D:\backup\对话记录\<日期>-完整对	323KB

	话.md	

10.4 一句话总结

会话是短期记忆，记忆文件是长期记忆；裁剪只动前者，备份兜底全部。

按 turn 整段裁剪、重排 seq/turn、抢救状态事件、删掉有状态的运行时日志、清投影缓存、重启后验证——六步缺一不可。

报告完 · 2026-10-26